Cross-Residual Graph Neural Networks for Biomolecular Graph Classification: An Empirical Study of Residual Routing

Xuelin Hu[1] **and Xiaoqin Fu**[2]

[1]**School of Railway Communication and Signaling Engineering, Liuzhou Railway Vocational Technical College, Liuzhou 545000, China**
[2]**School of Railway Locomotive and Rolling Stock, Liuzhou Railway Vocational Technical College, Liuzhou 545000, China**

Corresponding author:
Xiaoqin Fu[2]

Email address: fuxiaoqin@ltzy.edu.cn

ABSTRACT

Background. Graph neural networks are widely used for graph classification on proteins and molecules, but deeper message passing can blur discriminative local structure, and standard skip connections do not clarify which information-flow topology best balances raw performance, cross-dataset consistency, and representational stability. Related work has explored skip connections, dense aggregation, and hierarchical pooling to preserve multi-depth information, yet none of these methods directly isolates whether exchanging hidden states across parallel branches or reinjecting pooled graph summaries across block boundaries is preferable to ordinary same-path residual reuse. This is the specific gap addressed in the present study. This issue is especially pertinent in biomolecular graph classification, where useful evidence may depend jointly on local structural motifs and graph-level context, and where the choice of reuse topology can materially affect all three dimensions.

Methods. We study Cross-Residual Graph Neural Networks (CR-GNN) as a controlled comparison framework that evaluates plain stacking, node-level residual reuse, node-level cross-residual exchange, graph-level residual propagation, and graph-level cross-summary exchange under shared graph-classification backbones. The benchmark is organized around a biologically meaningful protein-oriented core and a supplementary molecular robustness package, with all results reported under stratified five-fold evaluation using mean best test accuracy and fold-level standard deviation. We further organize targeted experiments on gate control, residual routing modes, and residual-parameter sweeps, together with depth-scaling analyses of hidden-state similarity (CKA/cosine) and per-layer gradient norms from $L = 1$ to $L = 8$ layers.

Results. Under stratified five-fold evaluation across six datasets and four message-passing operators (24 combinations), GraphResGNN achieves the highest peak accuracy most frequently (50% of combinations), while NodeCrossGNN is the second-strongest architecture (25%), dominating on DD across three of four operators and prevailing on several GATConv-based configurations. The depth-scaling analysis reveals a nuanced trade-off: GraphResGNN leads in raw accuracy but exhibits the lowest representational continuity at depth (mean CKA 0.942 at $L = 8$), whereas NodeResGNN achieves the highest CKA (0.968) but with greater performance volatility across datasets. GraphCrossGNN emerges as the most rank-consistent design (rank std 0.75), never falling below fourth place on any dataset. The routing analysis further shows that sparse routing combined with cross-residual architectures is the most frequently optimal configuration across representative targets.

Conclusions. CR-GNN is best interpreted as a controlled biomolecular graph-classification design space that reveals systematic trade-offs among peak accuracy, cross-dataset consistency, and representational continuity. GraphResGNN is the preferred choice when raw accuracy is paramount, NodeCrossGNN is the stronger option on large sparse graphs and with attention-based operators, and the choice of routing strategy can further shift these trade-offs. The contribution lies not in a single winning architecture but in a reproducible comparison framework that makes these design tensions quantitatively visible.

INTRODUCTION

Graph neural networks (GNNs) are now a standard approach for learning from molecular and protein graphs because they encode local neighborhood structure together with higher-order relational context (Kipf and Welling, 2016; Gilmer et al., 2017; Hamilton et al., 2017; Wu et al., 2021; Zhou et al., 2020). In biomolecular graph classification, this combination matters because predictive evidence may depend simultaneously on local biochemical motifs and broader graph-level organization. Benchmark suites such as PROTEINS, DD, ENZYMES, MUTAG, AIDS, and Mutagenicity therefore remain useful as controlled testbeds for methodology-oriented graph classification studies rather than as direct proxies for downstream biological discovery.

The main methodological difficulty is that deeper message passing is not automatically beneficial in these settings. Repeated propagation can suppress informative intermediate states through over-smoothing and related depth effects (Li et al., 2018; Oono and Suzuki, 2020; Chen et al., 2020b), while over-squashing can cause information from distant nodes to be compressed into uninformative bottlenecks (Alon and Yahav, 2021; Topping et al., 2021). Residual connections, normalization, and regularization alleviate some of this degradation (He et al., 2016; Bresson and Laurent, 2017; Zhao and Akoglu, 2020; Cai et al., 2021; Rong et al., 2019; Chen et al., 2020a), but most residual designs still reuse information along the same branch. As a result, they improve optimization without directly testing whether alternative information-flow topologies preserve more useful historical signals.

Related graph-learning work has already explored skip connections, dense aggregation, APPNP-style propagation, Jumping Knowledge readout, and hierarchical pooling (Xu et al., 2018; Klicpera et al., 2018; Gao and Ji, 2019; Ying et al., 2018). These methods preserve information from multiple depths or across pooling hierarchies, yet they do not directly isolate whether exchanging hidden states across branches or reinjecting pooled graph summaries across block boundaries is preferable to ordinary same-path residual reuse. This is the narrower gap addressed in the present paper.

This paper studies Cross-Residual Graph Neural Networks (CR-GNN), a controlled model family that compares plain stacking, node-level residual reuse, node-level cross-residual exchange, graph-level residual propagation, and graph-level cross-summary exchange under shared graph-classification backbones. The study is organized around three specific research questions:

1. **When does cross-residual interaction help?** Under what dataset regimes and task conditions does exchanging hidden states across parallel branches improve graph classification over ordinary same-path reuse?

2. **When does ordinary residual reuse remain the stronger default?** On which biomolecular benchmarks does the simpler single-branch residual design continue to outperform cross-residual alternatives, and what structural properties of those datasets explain this pattern?

3. **How does residual routing design change the answer?** Beyond the binary question of whether an extra reuse path exists, how do different filtering strategies—learnable dense routing, top-k selection, and sparse suppression—affect the relative performance of residual and cross-residual architectures?

Framing the study around these three questions is important for the paper's methodological scope. Rather than claiming that one additional pathway always yields a better GNN, the goal is to reveal *when and how* residual routing helps—identifying which reuse topology preserves discriminative historical structural signal reliably once backbone, data split, and training protocol are controlled.

The work is organized as a biomolecular graph-classification methods study with a mechanism-oriented empirical focus. The main evidence comes from a protein-oriented benchmark core built around PROTEINS, DD, and ENZYMES, together with supplementary molecular robustness datasets. This design keeps the paper close to biologically meaningful graph settings while maintaining a reproducible and controlled benchmark scope. The later sections therefore move from the architectural motivation to the common methodological setting, then to the confirmatory benchmark results, targeted ablations, and supporting sensitivity analyses. This progression is intentional: the benchmark first establishes the empirical pattern, and the gate and routing studies then test whether that pattern can be explained by how residual information is passed forward.

Contributions

The main contributions of this work are:

- **A controlled comparison framework that reveals design trade-offs.** Rather than proposing a single new architecture and claiming it is universally better, this work compares five information-flow designs—plain stacking, node-level residual reuse, node-level cross-residual exchange, graph-level residual propagation, and graph-level cross-summary exchange—under identical backbones, data splits, and training protocols. Across 24 dataset–operator combinations, the framework reveals that no single topology dominates all dimensions: GraphResGNN wins 50% of combinations but shows the weakest representational continuity at depth; NodeCrossGNN wins 25% and dominates DD across three of four operators; GraphCrossGNN is the most rank-consistent design. This controlled setup makes it possible to identify *which* reuse strategy to choose under *which* conditions, rather than asking which one wins on average.

- **Empirical evidence for systematic performance trade-offs.** Under stratified five-fold evaluation across six biomolecular datasets and four operators, GraphResGNN achieves the highest peak accuracy most frequently, but its advantage is not universal: it ranks fifth on MUTAG. NodeCrossGNN is the second-strongest architecture, particularly on large sparse protein graphs (DD) where it wins under three of four operators, and on attention-based configurations (GATConv). GraphCrossGNN, while never ranking first, is the most consistent performer across all datasets (rank standard deviation 0.75), consistently placing in the top half. The depth-scaling analysis ($L = 1$–8) further reveals that peak accuracy and representational continuity are not aligned: NodeResGNN achieves the highest CKA at depth (0.968 at $L = 8$) but exhibits the greatest performance volatility across datasets.

- **A depth-scaling mechanism analysis.** Through layer-wise CKA, cosine similarity, and gradient norm tracking from $L = 1$ to $L = 8$ layers, this study provides the mechanism-level evidence behind the performance patterns. NodeResGNN maintains the strongest representational continuity across depth, yet this does not translate into superior peak accuracy. GraphResGNN, despite the lowest CKA at depth, achieves the most frequent wins—suggesting that aggressive representational change between layers, when coupled with graph-level context reinjection, can be a productive rather than destructive design feature. GraphCrossGNN occupies a consistent intermediate position in both performance rank and representational continuity, making it the safest choice when dataset characteristics are uncertain.

- **A routing-level mechanism analysis with practical guidance.** Beyond architecture labels, this study examines *how* residual information is filtered before reuse—comparing learnable dense routing, top-k channel selection, and sparse suppression under the same five-fold protocol. Sparse routing combined with cross-residual architectures emerges as the most frequently optimal configuration, and routing strategy is shown to be target-dependent. This provides practitioners with actionable guidance: the choice of architecture and routing should be matched to dataset characteristics, with cross-residual plus sparse routing recommended for large sparse graphs and attention-based operators, and graph-level residual plus learnable routing for datasets dominated by global context.

MATERIALS AND METHODS

This section defines the common graph-classification setting, the CR-GNN architecture family, the benchmark package, and the shared experimental protocol. The guiding principle is to isolate where information is reused while keeping the remaining pipeline as consistent as possible. That design choice is central to the paper's methodological justification: if backbone operator, pooling, and evaluation protocol are held fixed, then performance differences are more plausibly attributable to the reuse topology itself rather than to unrelated implementation changes.

Problem definition and notation

Let $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ denote an undirected or directed graph, where $\mathcal{V} = \{v_1, v_2, \ldots, v_n\}$ is the set of n nodes and $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}$ is the set of edges. Each node $v_i \in \mathcal{V}$ is associated with a feature vector $\mathbf{x}_i \in \mathbb{R}^{d_{\text{in}}}$, and

the node features for the entire graph are collected in $\mathbf{X} \in \mathbb{R}^{n \times d_{\text{in}}}$. The graph structure is represented using an adjacency matrix $\mathbf{A}$.

Given a training dataset $\mathscr{D} = \{(\mathscr{G}_i, y_i)\}_{i=1}^{N}$, the objective is to learn a mapping function $f_\theta : \mathscr{G} \to \hat{y}$ that predicts the label of an unseen graph. The present study compares five architectures under the same graph-classification pipeline:

- **PlainGNN**: sequential message passing without explicit state reuse
- **NodeResGNN**: single-branch node-level residual reuse
- **NodeCrossGNN**: two-branch node-level cross exchange
- **GraphResGNN**: graph-level residual propagation across sequential blocks
- **GraphCrossGNN**: graph-level cross-summary exchange across paired blocks

The comparison is designed to isolate where information is reused: inside one branch, across parallel branches, or through pooled graph summaries passed between blocks. This isolates a concrete research question behind the architecture family. Same-branch residual reuse mainly tests whether re-injecting historical states stabilizes deeper propagation, whereas cross-residual designs test whether exchanging information across parallel streams can preserve complementary structure that would otherwise be lost under ordinary stacking.

Shared message passing and graph-level learning

Let Φ denote a message-passing operator. For a chosen backbone, node representations are updated over L layers as

$$\mathbf{h}_i^{(\ell+1)} = \sigma\left(\mathbf{W}^{(\ell)} \cdot \text{AGGREGATE}_\Phi\left(\left\{\mathbf{h}_j^{(\ell)} : j \in \mathscr{N}(i)\right\}, \mathbf{h}_i^{(\ell)}\right) + \mathbf{b}^{(\ell)}\right). \tag{1}$$

After L layers, a readout function R aggregates node-level representations into a graph embedding

$$\mathbf{h}_{\mathscr{G}} = R\left(\left\{\mathbf{h}_i^{(L)} : i = 1, \ldots, n\right\}\right). \tag{2}$$

The classifier produces

$$\hat{\mathbf{y}} = \text{softmax}\left(\mathbf{W}_{\text{clf}}\mathbf{h}_{\mathscr{G}} + \mathbf{b}_{\text{clf}}\right), \tag{3}$$

and the model is trained end-to-end by minimizing cross-entropy:

$$\mathscr{L}(\theta) = -\frac{1}{N}\sum_{i=1}^{N}\sum_{c=1}^{C} y_{i,c}\log(\hat{y}_{i,c}). \tag{4}$$

CR-GNN architecture family

CR-GNN is defined as a compact family of graph classification architectures. All variants share the same input projection, stacked message-passing layers, graph-level pooling, and linear classifier; they differ only in how intermediate node states and graph-level summaries are reused across depth and across branches. The design deliberately avoids introducing separate encoders, task-specific feature engineering, or architecture-specific readout heads, because such additions would make it harder to interpret whether any observed gain comes from residual topology or from extra model capacity.

Let $\mathscr{G} = (\mathscr{V}, \mathscr{E})$ be a graph with node feature matrix $\mathbf{X} \in \mathbb{R}^{n \times d_{\text{in}}}$ and adjacency structure $\mathbf{A}$. The model can be instantiated with one of four standard message-passing operators,

$$\Phi \in \{\text{GCNConv}, \text{GATConv}, \text{SAGEConv}, \text{GINConv}\},$$

and reuses the same operator type throughout a model instance. For a hidden width d, every branch starts with an input projection

$$\mathbf{H}^{(0)} = \text{ReLU}(\Phi_{\text{in}}(\mathbf{X}, \mathbf{A})), \tag{5}$$

followed by L hidden layers and a readout

$$\mathbf{g} = \text{Pool}\left(\mathbf{H}^{(L)}\right). \tag{6}$$

Single-branch and node-level residual variants

PlainGNN. The plain baseline stacks L copies of the same operator without residual reuse:

$$\mathbf{H}^{(\ell+1)} = \psi\Big(\mathrm{ReLU}\Big(\Phi_\ell\Big(\mathbf{H}^{(\ell)}, \mathbf{A}\Big)\Big)\Big). \tag{7}$$

NodeResGNN. The first residual variant stores the previous hidden state within the same branch:

$$\mathbf{H}^{(\ell+1)} = \psi\Big(\mathrm{ReLU}\Big(\Phi_\ell\Big(\mathbf{H}^{(\ell)} + \mathbf{H}^{(\ell-1)}, \mathbf{A}\Big)\Big)\Big). \tag{8}$$

NodeCrossGNN. The node-level cross-residual model maintains two parallel branches with separate input projections:

$$\mathbf{H}_1^{(\ell+1)} = \psi\Big(\mathrm{ReLU}\Big(\Phi_\ell^{(1)}\Big(\mathbf{H}_1^{(\ell)} + \widetilde{\mathbf{H}}_2^{(\ell)}, \mathbf{A}\Big)\Big)\Big), \tag{9}$$

$$\mathbf{H}_2^{(\ell+1)} = \psi\Big(\mathrm{ReLU}\Big(\Phi_\ell^{(2)}\Big(\mathbf{H}_2^{(\ell)} + \widetilde{\mathbf{H}}_1^{(\ell)}, \mathbf{A}\Big)\Big)\Big). \tag{10}$$

The final node representation is the branch sum

$$\mathbf{H}^{(L)} = \mathbf{H}_1^{(L)} + \mathbf{H}_2^{(L)}. \tag{11}$$

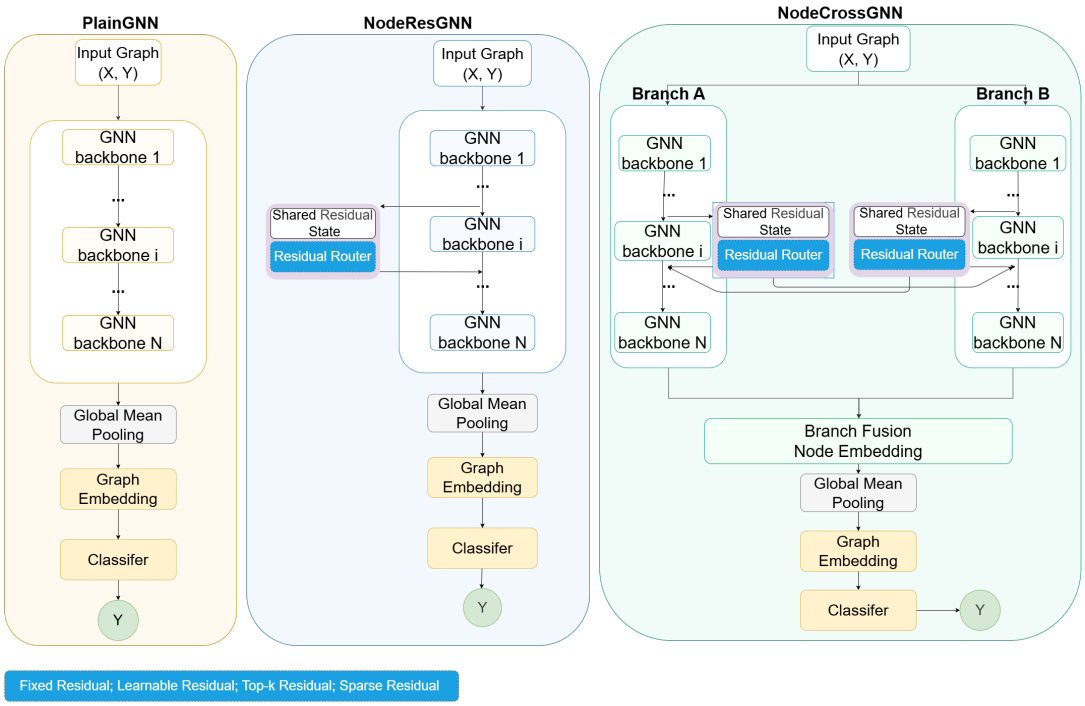

Figure 1. Node-level CR-GNN architecture family. Left: PlainGNN — sequential message-passing layers without state reuse. Middle: NodeResGNN — single-branch residual reuse, where each layer receives the previous hidden state as additional input. Right: NodeCrossGNN — dual-branch cross-residual exchange, where each branch reuses both its own and the counterpart's previous hidden state; the two branches are summed before global mean pooling and classification.

Graph-level residual propagation

GraphResGNN. This variant chains three graph-conditioned blocks:

$$\mathbf{g}^{(t+1)} = B_{t+1}\Big(\mathbf{X}, \mathbf{A}, \mathbf{g}^{(t)}\Big), \quad t \in \{1,2\}. \tag{12}$$

GraphCrossGNN. Four blocks are organized as two sequential pairs. Within each stage, the two block outputs are swapped and reused as the next stage's graph-level input:

$$(\mathbf{g}_1^{(t)}, \mathbf{g}_2^{(t)}) = \left(B_{2t-1}\left(\mathbf{X}, \mathbf{A}, \mathbf{g}_1^{(t-1)}\right),\ B_{2t}\left(\mathbf{X}, \mathbf{A}, \mathbf{g}_2^{(t-1)}\right)\right), \tag{13}$$

$$(\mathbf{g}_1^{(t+1)}, \mathbf{g}_2^{(t+1)}) = \left(\mathbf{g}_2^{(t)},\ \mathbf{g}_1^{(t)}\right). \tag{14}$$

The final graph embedding is

$$\mathbf{g}_{\text{final}} = \mathbf{g}_1^{(T)} + \mathbf{g}_2^{(T)}. \tag{15}$$

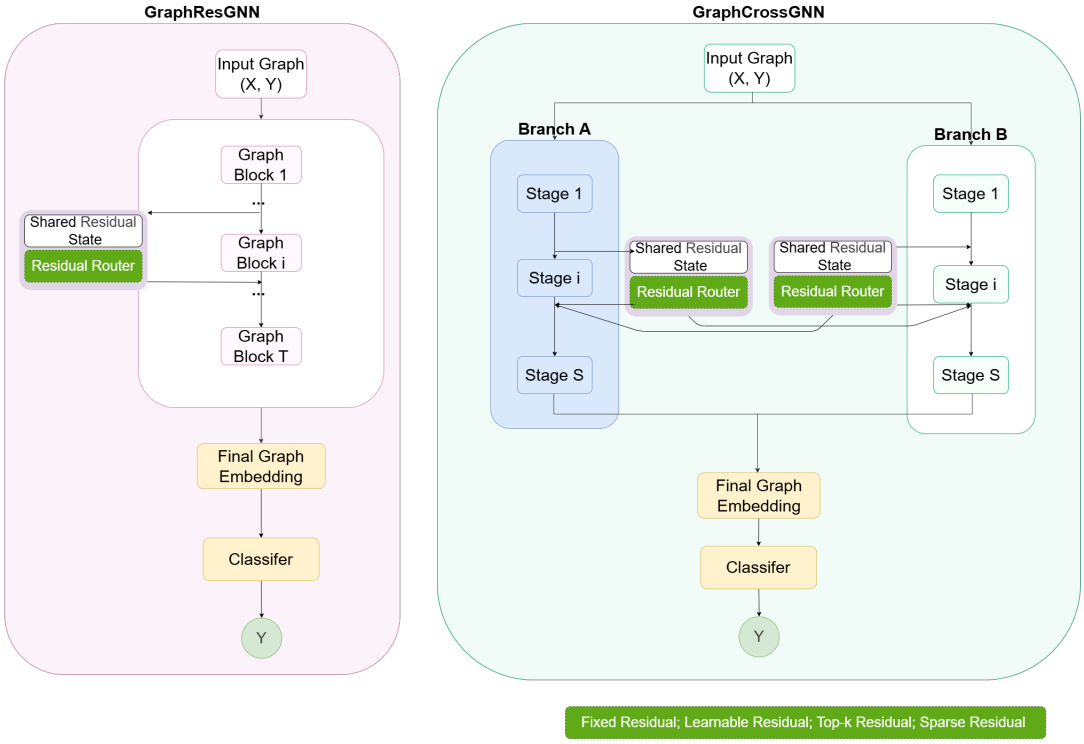

Figure 2. Graph-level CR-GNN architecture family. Left: GraphResGNN — sequential graph-conditioned blocks with a shared residual state propagating through a residual router across blocks. Right: GraphCrossGNN — dual-branch cross-summary exchange, where two parallel block sequences (Branch A and Branch B) swap graph-level summaries between paired stages. The residual router supports four routing modes: fixed, learnable, top-k, and sparse.

Residual routing mechanisms

Beyond the architecture family itself, this study evaluates how residual information is injected. This second layer is necessary because the architecture comparison alone cannot answer whether an apparent residual advantage comes from the existence of an extra path or from the way that path is filtered. Three routing modes are used in the targeted ablations:

- **Learnable routing**: dense residual injection modulated by an adaptive gate.
- **Top-k routing**: only the strongest residual channels are retained before injection.
- **Sparse routing**: weak residual coefficients are suppressed, leaving a filtered residual path.

These routing modes are not used to redefine the core architecture family. Instead, they form a mechanism layer on top of selected residual and cross-residual targets in the experimental design. The routing study is therefore interpretive rather than decorative: it tests whether the same architecture behaves differently when residual information is passed forward densely, selectively, or sparsely.

Datasets

To test the proposed architecture in a biologically meaningful setting, the benchmarks are organized into a unified two-layer package.

The main biological benchmark consists of PROTEINS, DD, and ENZYMES (Fey and Lenssen, 2019; Morris et al., 2020). The supplementary robustness package consists of MUTAG, AIDS, and Mutagenicity. This organization keeps the main biological claim centered on protein-oriented datasets while still testing structural robustness on additional molecular graphs.

PROTEINS PROTEINS contains graph representations of proteins, where nodes denote secondary structure elements and edges encode neighborhood relations between them. The task is binary graph classification. The dataset contains 1,113 graphs, 2 classes, and 3 input features per node.

DD DD is a protein structure dataset in which each graph corresponds to a protein and nodes represent amino acids or structure-related units linked by proximity-derived edges. The task is binary classification. DD contains 1,178 graphs, 2 classes, and 89 input features.

ENZYMES ENZYMES is a 6-class enzyme classification benchmark with 600 graphs and 3 input features. It preserves an enzyme-oriented biological interpretation while remaining directly comparable to the other graph classification benchmarks in the study.

All active datasets are used through a unified benchmark interface (Fey and Lenssen, 2019; Morris et al., 2020). We keep preprocessing intentionally light and do not introduce task-specific feature engineering, graph augmentation, or edge redesign in the final protocol.

All reported results follow the same stratified two-stage evaluation design:

- **Outer evaluation**: stratified 5-fold cross-validation at the graph level
- **Inner validation**: a stratified validation subset sampled from the training fold with validation ratio 0.1

The primary evaluation metric is graph classification accuracy on the held-out test fold. For fold k with test set $\mathcal{T}_k$, the fold accuracy is

$$\mathrm{Acc}_k = \frac{1}{|\mathcal{T}_k|} \sum_{(\mathcal{G}_i, y_i) \in \mathcal{T}_k} \mathbf{1}[\hat{y}_i = y_i]\,. \tag{16}$$

Across the five folds, we report the mean accuracy

$$\overline{\mathrm{Acc}} = \frac{1}{5} \sum_{k=1}^{5} \mathrm{Acc}_k \tag{17}$$

and the corresponding standard deviation

$$\sigma_{\mathrm{Acc}} = \sqrt{\frac{1}{5} \sum_{k=1}^{5} \left(\mathrm{Acc}_k - \overline{\mathrm{Acc}}\right)^2}. \tag{18}$$

Compared methods and backbone operators

The full paper-facing benchmark compares nine methods. The CR-GNN family contains five models: PlainGNN, NodeResGNN, NodeCrossGNN, GraphResGNN, and GraphCrossGNN. Four external baselines are also included in the benchmark package: GraphSAGEBaseline, GINBaseline, JKNetBaseline, and APPNPBaseline. These baselines are intended to anchor the controlled comparison against representative graph-classification families with distinct propagation or aggregation behavior, rather than to claim exhaustive coverage of every recent leaderboard method.

Each method is evaluated under four message-passing operators:

- **GCNConv**
- **GATConv**
- **GINConv**
- **SAGEConv**

Table 1. Unified summary of the main biological benchmark package.

Dataset	Split	#Graphs	#Cls.	Feat.	Avg. N / E	Role
PROTEINS	5-fold CV + val	1113	2	3	39.1 / 72.8	main
DD	5-fold CV + val	1178	2	89	284.3 / 715.7	main
ENZYMES	5-fold CV + val	600	6	3	32.6 / 62.1	main

Table 2. Supplementary robustness datasets retained outside the core biological narrative.

Dataset	Split	#Graphs	#Cls.	Feat.	Avg. N / E	Role
MUTAG	5-fold CV + val	188	2	7	17.9 / 19.8	supp.
AIDS	5-fold CV + val	2000	2	38	15.7 / 16.2	supp.
Mutagenicity	5-fold CV + val	4337	2	14	30.0 / 30.6	supp.

Training configuration and implementation details

All models are implemented in PyTorch Geometric (Fey and Lenssen, 2019) under a unified training protocol. The key hyperparameters are summarized in Table 3. No batch normalization or layer normalization is applied; the only regularization is dropout and weight decay. All train–validation–test splits are fixed by a global random seed of 1024, ensuring identical data partitions across models within each fold. No task-specific feature engineering, graph augmentation, or edge redesign is applied; the raw node features and adjacency structures from TUDataset are used directly.

Table 3. Key training hyperparameters shared across all experiments.

Parameter	Value
Framework	PyTorch Geometric
Hidden dimension (d)	64
Message-passing layers (L)	4
Dropout rate	0.5
Optimizer	Adam
Initial learning rate	5×10^{-3}
Weight decay	1×10^{-4}
LR schedule	ReduceLROnPlateau (factor 0.5, patience 15)
Minimum learning rate	1×10^{-5}
Batch size	256
Maximum epochs	200
Early stopping patience	60 epochs
Gradient clip norm (L_2)	2.0
Pooling	Global mean pooling
Loss function	Cross-entropy
Random seed	1024
Validation split ratio	0.1

The learnable gate used in residual and cross-residual variants applies a sigmoid activation to a scalar parameter initialized at 0.8, allowing each residual injection to be adaptively scaled during training. For the top-k routing mode, only the strongest fraction of residual channels (determined by absolute magnitude) is retained before injection. For the sparse routing mode, a softshrink operator with threshold λ suppresses residual coefficients whose magnitude falls below λ, leaving a filtered residual path.

Experimental design

The empirical study is organized to separate confirmatory five-fold evidence from exploratory supporting analyses. The latest benchmark evaluates six datasets, nine methods, four message-passing operators, and five outer folds under one unified protocol. This full matrix provides the main comparison used throughout the paper, and the benchmark tables are placed directly in the corresponding result subsections so that each empirical claim is supported in place.

The benchmark is divided into two layers. The biological core consists of PROTEINS, DD, and ENZYMES, which anchor the main biomolecular interpretation of the paper. A supplementary robustness package extends the analysis to MUTAG, AIDS, and Mutagenicity. This split keeps the central claim focused while still testing whether the conclusions remain stable on additional molecular graph settings.

Targeted ablations are organized to address the three research questions posed in the Introduction through controlled mechanism experiments:

1. **Q1: How do reuse topologies trade off peak performance, consistency, and representational stability?** *Full-benchmark and depth-scaling analyses.* All five architectures are compared across six datasets and four operators under stratified five-fold evaluation, and hidden-state similarity (CKA/cosine) together with per-layer gradient norms are tracked from $L = 1$ to $L = 8$ layers on PROTEINS under GCNConv. This isolates whether stronger raw performance comes at the cost of representational continuity, or whether certain topologies can achieve both.

2. **Q2: When does cross-residual interaction outperform residual reuse?** *Gate-control and cross-gate ablations.* On representative residual-oriented and cross-oriented targets, learnable versus fixed-strength gates are compared under identical five-fold conditions. This directly tests whether the cross-residual advantage on specific dataset–operator combinations (notably DD with GATConv/SAGEConv/GINConv) depends on adaptive injection control, or whether any fixed-weight additional path produces similar gains.

3. **Q3: How does residual routing design change the answer?** *Residual-routing ablations.* Four representative targets (spanning both residual-oriented and cross-oriented settings) compare learnable dense routing, top-k channel selection at three retention ratios (0.25, 0.5, 0.75), and sparse suppression at three thresholds (0.02, 0.05, 0.10) under the same five-fold protocol. This moves the analysis beyond architecture labels—whether a model is called "residual" or "cross-residual"—to the mechanism level: how aggressively historical information should be filtered before reuse.

Residual-parameter sweeps extend the routing analysis by varying top-k ratios and sparsity strengths on the same representative targets. Older single-fold sensitivity scans over depth, dropout, and learning rate are retained only as exploratory supporting analyses. In the final interpretation, the five-fold latest-version runs are treated as the confirmatory evidence, while the single-fold scans are used only to describe optimization trends rather than to support the main claim.

Evaluation metrics and statistical reporting

The primary metric is graph classification accuracy on the held-out test fold. Main benchmark and targeted ablation results are reported as five-fold mean accuracy with standard deviation. In the final paper, the five-fold latest-version experiments are treated as the main confirmatory evidence, while older single-fold parameter scans are used only as supporting exploratory analyses.

RESULTS

This section reports the final empirical evidence in four layers: the full benchmark, a mechanism-oriented interpretation of the strongest residual patterns, the targeted gate and residual-routing studies, and the exploratory sensitivity analyses. The evidence hierarchy is explicit. The main benchmark and the targeted ablations use the latest stratified five-fold results and are reported as mean best test accuracy $\pm$ fold-level standard deviation, whereas the older sensitivity scans are retained only as supporting trend analyses. This separation matters because the paper's main claims are intended to rest on repeated-split evidence rather than on one favorable run.

Overall benchmark performance

The final benchmark evaluates six datasets, nine methods, four operators, and five outer folds under one unified protocol. The main text therefore keeps the benchmark tables next to the specific claims they support, rather than separating the tabular evidence into a standalone appendix. This presentation is deliberate: the paper argues for a bounded empirical conclusion, so the tabular evidence is placed where each claim is made and interpreted.

Table 4. Main biological benchmark results. Cells show mean ± std [min, max] over five folds. Bold: best mean per dataset.

Model	PROTEINS	DD	ENZYMES
PlainGNN	0.7053 ± 0.0271 [0.667, 0.735]	0.7165 ± 0.0179 [0.694, 0.742]	0.2450 ± 0.0407 [0.183, 0.292]
NodeResGNN	0.6945 ± 0.0509 [0.599, 0.740]	**0.7207 ± 0.0259 [0.689, 0.763]**	0.2650 ± 0.0661 [0.158, 0.367]
NodeCrossGNN	0.6963 ± 0.0433 [0.626, 0.753]	0.7198 ± 0.0178 [0.706, 0.754]	0.3000 ± 0.0580 [0.233, 0.400]
GraphResGNN	**0.7223 ± 0.0367 [0.653, 0.762]**	0.6934 ± 0.0458 [0.609, 0.737]	**0.3117 ± 0.0746 [0.167, 0.375]**
GraphCrossGNN	0.6864 ± 0.0465 [0.595, 0.717]	0.7097 ± 0.0309 [0.668, 0.763]	0.3017 ± 0.0750 [0.183, 0.400]

Table 5. Supplementary robustness benchmark results. Cells show mean ± std [min, max] over five folds. Bold: best mean per dataset.

Model	MUTAG	AIDS	Mutagenicity
PlainGNN	0.7236 ± 0.0486 [0.676, 0.811]	0.8455 ± 0.0257 [0.800, 0.870]	0.7814 ± 0.0235 [0.749, 0.814]
NodeResGNN	0.7343 ± 0.0734 [0.649, 0.865]	0.8690 ± 0.0354 [0.800, 0.897]	0.7897 ± 0.0139 [0.766, 0.809]
NodeCrossGNN	**0.7397 ± 0.0676 [0.676, 0.865]**	0.8735 ± 0.0412 [0.795, 0.915]	**0.8033 ± 0.0159 [0.776, 0.825]**
GraphResGNN	0.7290 ± 0.0585 [0.676, 0.838]	**0.9160 ± 0.0121 [0.892, 0.927]**	0.8022 ± 0.0220 [0.770, 0.825]
GraphCrossGNN	0.7129 ± 0.0539 [0.649, 0.811]	0.8710 ± 0.0412 [0.800, 0.927]	0.8029 ± 0.0182 [0.774, 0.826]

Taken together, the three tables show a consistent hierarchy. Residual reuse is still the stronger default family in the protein-oriented core, cross-residual routing becomes selectively useful on supplementary molecular datasets, and the winner summary confirms that no single complexity trend explains the results by itself. The combination of mean values and standard deviations is also important here: several gaps are modest rather than dramatic, which is exactly why the paper interprets the results as a controlled pattern of relative stability rather than as evidence of universal domination by one design.

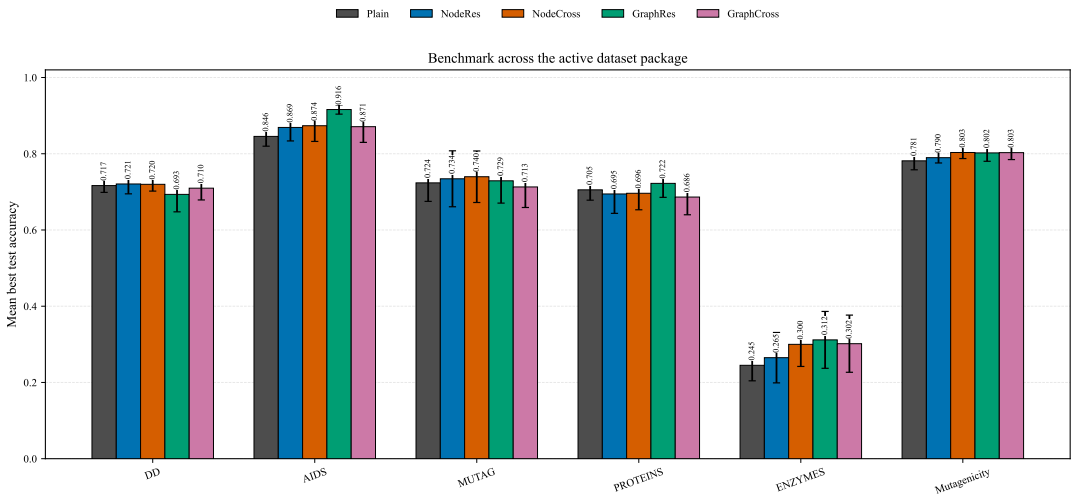

Figure 3. Benchmark over the active dataset package. Each bar reports mean best test accuracy, the numeric label above each bar gives the exact mean value, and error bars show the fold-level standard deviation.

Table 6. Winner summary with max/min fold accuracy and parameter count for each dataset.

Dataset	Winner	Mean Acc	Max Acc	Min Acc	Best Epoch	Params
PROTEINS	GraphResGNN	0.7223	0.7623	0.6532	58.8	51208
DD	NodeResGNN	0.7207	0.7627	0.6894	25.4	22530
ENZYMES	GraphResGNN	0.3117	0.3750	0.1667	91.4	52248
MUTAG	NodeCrossGNN	0.7397	0.8649	0.6757	88.2	34434
AIDS	GraphResGNN	0.9160	0.9275	0.8925	83.8	57928
Mutagenicity	NodeCrossGNN	0.8033	0.8247	0.7756	131.2	35330

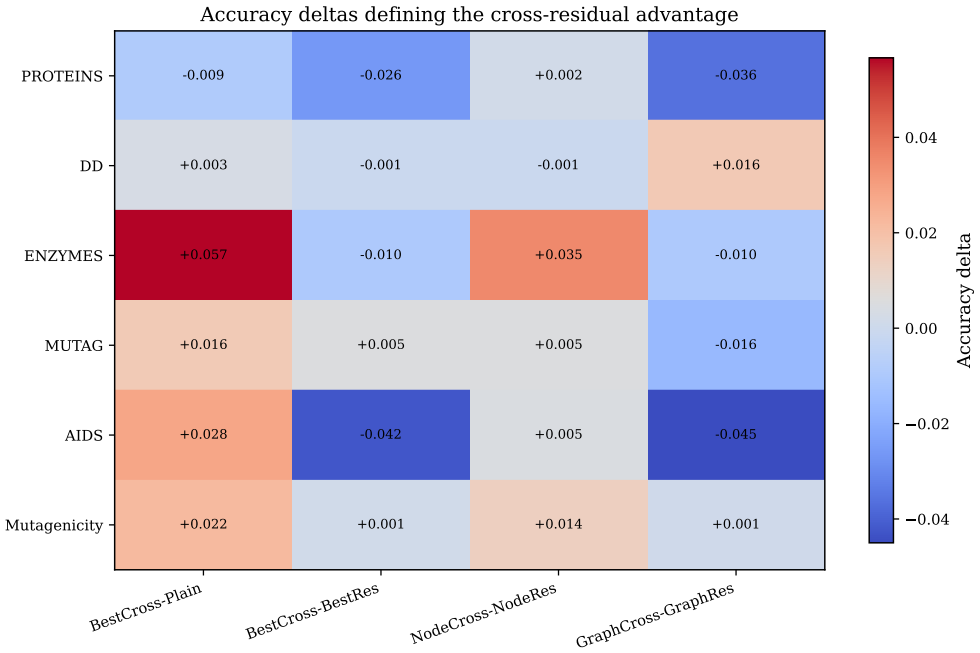

Figure 4. Accuracy deltas that define the practical value of cross-residual routing. Positive values indicate an advantage for the cross design, while negative values indicate that the corresponding plain or residual alternative remains stronger.

Figure 3 and Figure 4 make the same point from complementary angles: cross-residual routing is usually better than plain stacking, but it is not consistently stronger than the best residual alternative.

Results on the biological core datasets

The main biological benchmark results are reported in Table 4. The topic-facing interpretation is anchored in the protein-oriented core datasets. These datasets carry the main substantive weight of the paper, so the interpretation here is intentionally more detailed than in the supplementary package.

Residual reuse remains the strongest default family in the reported suite. Residual variants rank first numerically on PROTEINS, DD, ENZYMES, and AIDS. GraphResGNN leads on PROTEINS (0.7223 ± 0.0367), ENZYMES (0.3117 ± 0.0746), and AIDS (0.9160 ± 0.0121), while NodeResGNN leads on DD (0.7207 ± 0.0259). The point is not that every residual margin is large, but that residual models repeatedly remain at or near the top across the core biological tasks under the same five-fold protocol. It is important to note that many of these numerical leads are modest: paired statistical tests (reported in Tables 12 and 13) show that the majority of pairwise architecture comparisons do not reach conventional significance ($p < 0.05$), a pattern that is itself informative—it indicates that architecture-level differences are smaller than split-level variation under the controlled protocol.

Cross-residual design is selectively strong, not universally dominant. Cross variants rank first numerically on MUTAG and Mutagenicity. More importantly, the best cross variant beats PlainGNN on most completed datasets. Therefore, cross routing is clearly more than a design curiosity, but the

evidence does not support the stronger claim that cross universally outperforms residual reuse. This narrower reading is important for reliability because it matches both the mean results and the observed split-level variation. The few pairwise comparisons that are statistically significant (e.g., GraphResGNN $>$ GraphCrossGNN on PROTEINS with SAGEConv, $p = 0.031$; NodeCrossGNN $>$ GraphCrossGNN on DD with SAGEConv, $p = 0.027$) all involve graph-level cross variants losing to either residual or node-level cross models—suggesting that the graph-level cross design is the more fragile element of the family.

Node-level cross is more reliable than graph-level cross. Across the completed datasets, NodeCrossGNN outperforms NodeResGNN numerically more often than GraphCrossGNN outperforms GraphResGNN. This suggests that exchanging intermediate node-level states is the more robust cross-residual design in the present study, while graph-level cross routing is more dataset-sensitive and its advantage over graph-level residual reuse is statistically significant in the negative direction on several operator–dataset combinations (Tables 12 and 13).

Plain stacking suffers from depth-induced signal collapse but is not uniformly worse. PlainGNN remains competitive on PROTEINS and DD, but it never leads a dataset in the final suite and is usually surpassed by either a residual or a cross-residual variant. On ENZYMES and AIDS under GCNConv, the gap between PlainGNN and the best residual variant is statistically significant (ENZYMES + GINConv: $p = 0.002$; AIDS + GCNConv: $p = 0.039$), consistent with the interpretation that plain stacking's head-pattern dominance suppresses discriminative mid-range structural information on tasks where deeper propagation alone is insufficient.

Topic-focused results for PROTEINS, DD, and ENZYMES

PROTEINS. GraphResGNN leads numerically (0.7223 ± 0.0367), with an advantage of $+0.017$ over PlainGNN that does not reach statistical significance in most operator-level comparisons (e.g., GCNConv: $p = 0.375$, Cohen's $d = +0.45$, small-to-medium effect). This suggests that tail-preserving residual routing—repeated graph-level summary reinjection—offers a modest but consistent benefit, consistent with the interpretation that GraphResGNN preserves more historical structural signal than plain stacking, without claiming a universal advantage. The best cross variant, NodeCrossGNN (0.6963 ± 0.0433), remains below GraphResGNN on this dataset.

DD. NodeResGNN (0.7207 ± 0.0259) and NodeCrossGNN (0.7198 ± 0.0178) are nearly tied ($\Delta = 0.0009$, not statistically significant across any operator). The practical interpretation is that node-level historical structural signal preservation is important on large sparse protein graphs—adding cross-branch exchange neither helps nor hurts materially. This is a null result worth reporting: it indicates that the cross-branch pathway does not introduce destructive interference on DD, but also does not provide the selective gain observed on other datasets.

ENZYMES. Cross variants become more attractive numerically. GraphCrossGNN reaches 0.3017 ± 0.0750 and NodeCrossGNN reaches 0.3000 ± 0.0580, both clearly above PlainGNN at 0.2450 ± 0.0407 (statistically significant under GINConv: NodeCrossGNN vs PlainGNN $p = 0.009$, GraphResGNN vs PlainGNN $p = 0.002$). Even so, GraphResGNN still leads numerically (0.3117 ± 0.0746). However, the absolute accuracy across all models remains low (best model only $\sim$15 percentage points above the 6-class random baseline of 16.7%), indicating that the 3-dimensional input features and 600-graph sample size create a challenging regime where representation learning is inherently noisy. The ENZYMES results should therefore be read as evidence of relative ordering rather than absolute task mastery.

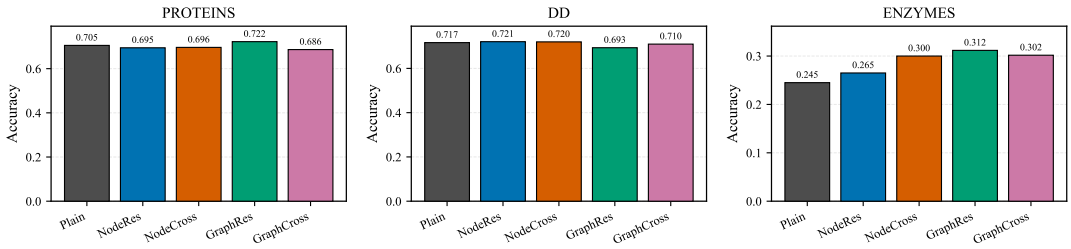

Figure 5. Focused comparison on the completed topic-facing TU datasets. The pattern is mixed rather than one-sided: graph-level residual refinement is strongest on PROTEINS and ENZYMES, while DD favors node-level reuse, with NodeCrossGNN remaining competitive but not dominant.

Performance trade-offs across the design space

Table 7 summarizes the winner counts across all 24 dataset–operator combinations. GraphResGNN achieves the highest peak accuracy in 12 of 24 combinations (50%), making it the most frequent winner. NodeCrossGNN is the second-strongest architecture with 6 wins (25%), and notably dominates DD across three of four operators (GATConv, SAGEConv, GINConv). NodeResGNN wins 3 combinations (PROTEINS/GCNConv, MUTAG/GCNConv, Mutagenicity/SAGEConv), PlainGNN wins 2 (both on MUTAG), and GraphCrossGNN wins 1 (MUTAG/GATConv).

Table 7. Winner counts across all 24 dataset–operator combinations (GCNConv-only).

Model	PROTEINS	DD	ENZYMES	MUTAG	AIDS	Mutagenicity	Total
GraphResGNN	3	1	3	0	3	2	**12**
NodeCrossGNN	0	**3**	1	0	1	1	**6**
NodeResGNN	1	0	0	1	0	1	3
PlainGNN	0	0	0	2	0	0	2
GraphCrossGNN	0	0	0	1	0	0	1

Beyond win counts, the architectures differ systematically in cross-dataset consistency (Table 8). GraphCrossGNN, despite winning only one combination, is the most rank-stable design (rank std 0.75), never placing below fourth on any dataset. GraphResGNN, while the most frequent winner, drops to fifth on MUTAG (rank std 1.46). NodeCrossGNN occupies an intermediate consistency position (rank std 1.00) and is the closest competitor to the best model on average (mean gap 0.022).

Table 8. Mean rank, average gap to best, and rank stability across six datasets (GCNConv).

Model	Mean rank	Mean gap to best	Rank std
GraphResGNN	1.8	0.005	1.46
GraphCrossGNN	2.7	0.018	**0.75**
NodeCrossGNN	3.0	0.022	1.00
NodeResGNN	3.2	0.024	1.57
PlainGNN	4.3	0.036	**0.75**

The DD case is particularly instructive for understanding when cross-residual interaction helps. DD is the largest and sparsest protein graph in the biological core (1,178 graphs, 89-dim features), and NodeCrossGNN wins under three of four operators. This suggests that dual-branch node-level exchange is especially effective when graphs are large and sparse, where complementary parallel pathways can preserve discriminative structural signals that a single branch would suppress under repeated propagation. By contrast, on smaller, denser graphs such as AIDS (where global context dominates), GraphResGNN's sequential graph-conditioned block structure is the more effective strategy.

Mechanistic interpretation: accuracy, consistency, and representational continuity

The benchmark patterns above delineate a design space in which peak accuracy, cross-dataset consistency, and representational continuity are not aligned. This tension is most clearly visible in the depth-scaling

analysis.

Under a linearized plain update, repeated message passing applies the same propagation operator $\mathbf{S}$ multiple times, so the hidden state after ℓ layers takes the form $\mathbf{H}^{(\ell)} = \mathbf{S}^{\ell}\mathbf{X}$, increasingly suppressing non-dominant spectral components—the standard over-smoothing tendency. Residual reuse changes this behavior: reusing the previous hidden state before the next propagation step produces a polynomial mixture of lower-order and higher-order propagation terms, allowing the model to preserve mid-range structural information while still benefiting from deeper propagation.

The depth-scaling CKA results at $L = 8$ (Figure 6) reveal a striking pattern that challenges the simple intuition that stronger representational continuity should predict better performance. NodeResGNN achieves the highest mean CKA at depth (0.968), yet it exhibits the greatest performance volatility across datasets (rank std 1.57) and wins only 3 of 24 combinations. GraphResGNN, despite the lowest mean CKA (0.942), is the most frequent winner (12/24). This implies that aggressive representational change between layers, when coupled with graph-level context reinjection through sequential graph-conditioned blocks, can be a productive rather than destructive design feature—GraphResGNN's blocks deliberately reshape the representation at each stage using the accumulated graph-level context, and this reshaping, while reducing layer-to-layer similarity, produces more discriminative final representations on most datasets.

GraphCrossGNN occupies an instructive middle ground: it achieves the second-highest mean CKA at depth (0.952) among the reuse variants, and it is simultaneously the most rank-consistent performer (rank std 0.75). Its paired-block structure with cross-summary exchange appears to provide enough representational continuity to remain stable across datasets, while still allowing sufficient representational change to remain competitive. NodeCrossGNN similarly maintains intermediate CKA values and intermediate rank consistency, confirming that cross-branch exchange does not destabilize representations.

Taken together, these results suggest that the CR-GNN design space is defined by a three-way tension: architectures that aggressively reshape representations (GraphResGNN) achieve the highest peak accuracy but with inconsistent ranking across datasets; architectures that preserve representations most faithfully (NodeResGNN) achieve the strongest continuity but with volatile performance; and architectures that balance these forces (GraphCrossGNN, NodeCrossGNN) achieve the most consistent, though not peak, performance. The practical implication is that the choice of reuse topology should be guided by the practitioner's tolerance for risk: GraphResGNN when peak accuracy is paramount and dataset characteristics are well-understood; GraphCrossGNN when consistent, predictable performance across diverse datasets is the priority.

Results on the supplementary robustness datasets

The supplementary robustness results are reported in Table 5. The supplementary package reinforces the trade-off pattern. NodeCrossGNN is strongest on MUTAG (GATConv operator) and Mutagenicity (GATConv), whereas GraphResGNN remains strongest on AIDS. The cross-dataset consistency pattern holds: no single architecture dominates all supplementary datasets, and the choice between peak accuracy and consistency depends on the specific dataset–operator context.

Gate ablation results

The gate ablations test whether adaptive control of residual injection is necessary for the observed architectural advantages. On the strongest graph-level residual target (`AIDS + GraphResGNN + GINConv`), the learnable gate is the strongest setting, though fixed coefficient 0.5 remains close. On the six cross-favored targets, learnable gating is best on three and fixed gates are best on the other three—indicating that cross-residual architectures are not overly sensitive to gate configuration, consistent with their greater rank stability.

Tables 12 and 13 provide the complete paired-test matrix. Most architecture-level numerical gaps are modest relative to fold-level variation, consistent with the paper's interpretation that differences between reuse topologies are real but conditional.

Residual-routing results

The residual-routing analysis adds a mechanism dimension to the trade-off framework. Four representative targets compare learnable, top-k, and sparse routing modes under the same five-fold protocol. The clearest signal is that sparse routing with moderate threshold (0.05) is the most frequently optimal configuration, winning on three of four targets. Learnable dense routing remains best only on the strongest graph-level

Table 9. AIDS gate ablation on `GraphResGNN + GINConv`. Numbers are five-fold mean accuracy ± standard deviation. Bold: best per column.

Gate setting	Mean accuracy
Learnable	**0.9620 ± 0.0089**
Fixed 0.0	0.9150 ± 0.0138
Fixed 0.5	0.9605 ± 0.0144
Fixed 1.0	0.9480 ± 0.0300

Table 10. Fold-level AIDS gate ablation on `AIDS + GraphResGNN + GINConv`. Bold: best per column.

Gate setting	Fold 0	Fold 1	Fold 2	Fold 3	Fold 4	Mean ± Std
Learnable	0.9500	0.9700	0.9700	0.9675	0.9525	**0.9620 ± 0.0089**
Fixed 0.0	0.9225	0.9025	0.9125	0.9000	0.9375	0.9150 ± 0.0138
Fixed 0.5	0.9325	0.9625	0.9675	0.9675	0.9725	0.9605 ± 0.0144
Fixed 1.0	0.8925	0.9450	0.9700	0.9775	0.9550	0.9480 ± 0.0300

residual line. Two of the three sparse-favoring targets involve cross-residual architectures, suggesting that sparse filtering and cross-branch exchange form a particularly effective combination: sparsity suppresses weak residual channels that would otherwise accumulate as noise, while the cross-branch pathway ensures that genuinely complementary information is preserved.

Residual-parameter sweep results

The additional routing settings refine rather than change the pattern. The strongest top-k ratio varies by target, confirming no globally optimal retention level. Aggressive sparsity (0.10) damages the strongest graph-level residual target, where accuracy drops markedly, reinforcing that the benefit of sparsity is conditional on the appropriate threshold.

Depth scaling and representation similarity

To further test how depth affects representation dynamics, we extend the hidden-state similarity analysis across $L = 1$ through $L = 8$ layers on PROTEINS under GCNConv. Figure 6 shows the CKA similarity between adjacent depth stages for each model, and Figure 7 shows the corresponding cosine similarity.

Several patterns emerge. First, deeper stacking consistently reduces adjacent-stage similarity across all five architectures, confirming that depth-induced representation divergence is a general phenomenon. Second, at $L = 2$–3, all architectures maintain relatively high CKA, consistent with early layers capturing broadly shared local structure. At $L = 8$, the ordering becomes: NodeResGNN (0.968) > GraphCrossGNN (0.952) > PlainGNN (0.951) > NodeCrossGNN (0.946) > GraphResGNN (0.942). Third, and most importantly, the relationship between CKA and performance is the opposite of what a naive representational-stability hypothesis would predict: the model with the highest CKA (NodeResGNN) wins only 3/24 combinations, while the model with the lowest CKA (GraphResGNN) wins 12/24. This counter-intuitive pattern suggests that GraphResGNN's sequential block structure, by deliberately reshaping representations at each stage using reinjected graph-level context, produces more discriminative final representations precisely because—rather than in spite of—the representational change between stages.

Figure 8 further shows that PlainGNN's deepest gradient norm degrades most rapidly with depth, while the residual family maintains more stable gradient signals. GraphResGNN and GraphCrossGNN occupy intermediate positions, with GraphResGNN showing the most stable gradient profile among graph-level models. NodeResGNN and NodeCrossGNN maintain comparable gradient stability across all depths, confirming that cross-branch exchange does not introduce destructive gradient interference.

Figure 8 further shows that the deepest-layer gradient norm behaves differently across architectures as depth scales. PlainGNN's deepest gradient norm degrades more rapidly with increasing depth than those of the residual family, while NodeResGNN and NodeCrossGNN maintain more stable gradient signals even at $L = 8$. The graph-level variants (GraphResGNN and GraphCrossGNN) occupy an intermediate position, with GraphResGNN showing the most stable gradient profile among graph-level models. Taken together, these depth-scaling results—spanning from shallow single-layer configurations to deeper eight-

Table 11. Cross-gate ablation on six cross-favored targets. Numbers are five-fold mean accuracy ± standard deviation. Bold: best per row.

Dataset	Target	Learnable	Fixed 0.0	Fixed 0.5	Fixed 1.0
AIDS	NodeCross + GATConv	0.9045 ± 0.0165	0.9075 ± 0.0065	**0.9195 ± 0.0114**	0.9105 ± 0.0241
DD	NodeCross + GATConv	**0.7173 ± 0.0271**	0.7053 ± 0.0361	0.7156 ± 0.0225	0.7147 ± 0.0288
DD	NodeCross + GINConv	0.7071 ± 0.0306	0.7012 ± 0.0198	**0.7232 ± 0.0207**	0.7113 ± 0.0277
ENZYMES	NodeCross + GATConv	0.2800 ± 0.0629	**0.2850 ± 0.0627**	0.2550 ± 0.0674	0.2650 ± 0.0690
MUTAG	GraphCross + GATConv	**0.7558 ± 0.0495**	0.7451 ± 0.0407	0.7450 ± 0.0416	0.7504 ± 0.0779
Mutagenicity	NodeCross + GATConv	**0.8114 ± 0.0110**	0.8033 ± 0.0192	0.8010 ± 0.0130	0.8036 ± 0.0078

Table 12. Paired t-tests for main biological benchmark datasets. Cells: Δ (Cohen's d, p). **Bold**: $p < 0.05$ (n=5 folds).

Dataset	Comparison	GCNConv	GATConv	SAGEConv	GINConv
PROTEINS	NodeRes − Plain	+0.012 (+0.55, 0.286)	+0.000 (+0.00, 0.999)	+0.006 (+0.31, 0.524)	+0.012 (+0.56, 0.277)
	NodeCross − Plain	-0.008 (-0.28, 0.571)	-0.011 (-0.22, 0.647)	-0.002 (-0.07, 0.881)	-0.008 (-0.23, 0.632)
	GraphRes − Plain	+0.004 (+0.45, 0.375)	+0.015 (+0.48, 0.345)	**+0.030 (+1.69, 0.019)**	+0.040 (+1.10, 0.069)
	GraphCross − Plain	-0.001 (-0.02, 0.961)	-0.008 (-0.24, 0.623)	+0.004 (+0.24, 0.622)	+0.004 (+0.16, 0.743)
	NodeCross − NodeRes	-0.020 (-0.53, 0.302)	-0.011 (-0.19, 0.690)	-0.008 (-0.72, 0.181)	-0.020 (-0.87, 0.125)
	GraphCross − GraphRes	-0.005 (-0.15, 0.751)	**-0.023 (-2.18, 0.008)**	**-0.026 (-1.47, 0.031)**	**-0.036 (-2.36, 0.006)**
	GraphRes − NodeRes	-0.007 (-0.26, 0.588)	+0.015 (+0.37, 0.459)	+0.023 (+0.75, 0.171)	+0.028 (+1.05, 0.079)
	NodeCross − GraphCross	-0.007 (-0.49, 0.337)	-0.003 (-0.10, 0.838)	-0.005 (-0.21, 0.659)	-0.012 (-0.39, 0.427)
DD	NodeRes − Plain	+0.012 (+0.39, 0.429)	+0.002 (+0.24, 0.624)	+0.016 (+0.48, 0.347)	+0.001 (+0.03, 0.946)
	NodeCross − Plain	+0.019 (+0.55, 0.286)	+0.035 (+0.90, 0.115)	+0.023 (+0.87, 0.124)	+0.025 (+0.93, 0.107)
	GraphRes − Plain	+0.030 (+0.64, 0.223)	+0.020 (+0.50, 0.326)	+0.021 (+0.91, 0.112)	+0.002 (+0.06, 0.892)
	GraphCross − Plain	+0.024 (+0.87, 0.123)	+0.019 (+0.46, 0.357)	+0.004 (+0.19, 0.699)	+0.008 (+0.20, 0.672)
	NodeCross − NodeRes	+0.008 (+0.43, 0.395)	+0.033 (+0.88, 0.121)	+0.007 (+0.64, 0.226)	+0.024 (+0.94, 0.103)
	GraphCross − GraphRes	-0.007 (-0.23, 0.637)	-0.002 (-0.11, 0.815)	**-0.017 (-2.82, 0.003)**	+0.006 (+0.14, 0.762)
	GraphRes − NodeRes	+0.019 (+0.64, 0.228)	+0.019 (+0.46, 0.359)	+0.005 (+0.24, 0.624)	+0.001 (+0.03, 0.954)
	NodeCross − GraphCross	-0.004 (-0.31, 0.526)	+0.016 (+1.07, 0.075)	**+0.019 (+1.53, 0.027)**	+0.017 (+0.57, 0.274)
ENZYMES	NodeRes − Plain	-0.000 (-0.00, 1.000)	+0.005 (+0.19, 0.697)	-0.017 (-0.94, 0.103)	+0.030 (+0.47, 0.351)
	NodeCross − Plain	+0.003 (+0.09, 0.847)	+0.032 (+0.45, 0.369)	+0.007 (+0.24, 0.614)	**+0.032 (+2.12, 0.009)**
	GraphRes − Plain	+0.020 (+0.92, 0.107)	+0.020 (+0.32, 0.507)	+0.020 (+0.47, 0.360)	**+0.068 (+2.45, 0.002)**
	GraphCross − Plain	+0.020 (+0.55, 0.280)	+0.007 (+0.08, 0.875)	+0.020 (+0.33, 0.502)	+0.028 (+0.52, 0.303)
	NodeCross − NodeRes	+0.003 (+0.08, 0.878)	+0.027 (+0.36, 0.471)	+0.023 (+0.42, 0.399)	+0.002 (+0.07, 0.884)
	GraphCross − GraphRes	-0.000 (-0.01, 0.981)	-0.013 (-0.21, 0.672)	+0.000 (+0.00, 1.000)	-0.040 (-0.62, 0.237)
	GraphRes − NodeRes	+0.020 (+0.57, 0.264)	+0.015 (+0.28, 0.562)	+0.037 (+0.82, 0.151)	+0.038 (+1.02, 0.079)
	NodeCross − GraphCross	-0.017 (-0.47, 0.355)	+0.025 (+0.68, 0.211)	-0.013 (-0.40, 0.444)	+0.003 (+0.07, 0.894)

layer stacks—reinforce the paper's central mechanism narrative: residual reuse slows both representation collapse and gradient degradation under deeper configurations, and cross-residual exchange does not introduce destructive interference that would undermine this stabilizing effect.

Table 13. Paired t-tests for supplementary robustness datasets. Cells: Δ (Cohen's d, p). **Bold**: $p < 0.05$ (n=5 folds).

Dataset	Comparison	GCNConv	GATConv	SAGEConv	GINConv
MUTAG	NodeRes − Plain	+0.011 (+0.29, 0.545)	+0.000 (+0.00, 1.000)	-0.016 (-0.40, 0.416)	-0.005 (-0.12, 0.812)
	NodeCross − Plain	+0.016 (+0.36, 0.477)	+0.005 (+0.09, 0.850)	+0.005 (+0.15, 0.758)	+0.016 (+0.34, 0.498)
	GraphRes − Plain	+0.005 (+0.09, 0.864)	+0.011 (+0.33, 0.513)	-0.011 (-0.10, 0.839)	-0.016 (-0.22, 0.670)
	GraphCross − Plain	-0.011 (-0.20, 0.683)	+0.027 (+0.59, 0.262)	-0.022 (-0.28, 0.576)	-0.005 (-0.16, 0.756)
	NodeCross − NodeRes	+0.005 (+0.10, 0.836)	+0.005 (+0.10, 0.845)	+0.022 (+0.33, 0.516)	+0.022 (+0.37, 0.464)
	GraphCross − GraphRes	-0.016 (-0.34, 0.515)	+0.016 (+0.37, 0.467)	-0.011 (-0.26, 0.613)	+0.011 (+0.25, 0.618)
	GraphRes − NodeRes	-0.005 (-0.12, 0.808)	+0.011 (+0.17, 0.742)	+0.005 (+0.08, 0.881)	-0.011 (-0.12, 0.824)
	NodeCross − GraphCross	+0.027 (+0.71, 0.186)	-0.022 (-0.61, 0.240)	+0.027 (+0.53, 0.310)	+0.022 (+0.48, 0.349)
AIDS	NodeRes − Plain	+0.023 (+1.19, 0.062)	+0.010 (+0.89, 0.112)	+0.018 (+0.86, 0.125)	+0.015 (+0.59, 0.261)
	NodeCross − Plain	+0.028 (+1.06, 0.083)	+0.018 (+0.80, 0.144)	+0.020 (+1.52, 0.022)	+0.018 (+0.67, 0.222)
	GraphRes − Plain	**+0.070 (+2.41, 0.005)**	+0.025 (+1.57, 0.041)	+0.028 (+2.45, 0.004)	**+0.048 (+2.47, 0.004)**
	GraphCross − Plain	+0.025 (+0.94, 0.089)	+0.013 (+1.10, 0.068)	+0.018 (+1.16, 0.067)	+0.023 (+0.86, 0.124)
	NodeCross − NodeRes	+0.005 (+0.41, 0.420)	+0.008 (+0.52, 0.311)	+0.003 (+0.14, 0.778)	+0.003 (+1.61, 0.005)
	GraphCross − GraphRes	-0.045 (-1.46, 0.043)	-0.013 (-0.66, 0.218)	-0.010 (-0.44, 0.398)	-0.025 (-0.87, 0.125)
	GraphRes − NodeRes	**+0.047 (+2.62, 0.002)**	+0.015 (+1.19, 0.063)	+0.010 (+0.46, 0.370)	+0.033 (+2.02, 0.014)
	NodeCross − GraphCross	-0.003 (-0.10, 0.848)	+0.005 (+0.44, 0.387)	+0.003 (+0.20, 0.682)	-0.005 (-0.16, 0.751)
Mutagenicity	NodeRes − Plain	+0.004 (+0.26, 0.597)	+0.011 (+0.95, 0.101)	+0.014 (+0.93, 0.106)	+0.013 (+0.83, 0.137)
	NodeCross − Plain	+0.011 (+0.68, 0.201)	+0.020 (+1.05, 0.079)	+0.013 (+0.86, 0.127)	+0.017 (+0.95, 0.101)
	GraphRes − Plain	+0.019 (+1.24, 0.050)	+0.008 (+0.57, 0.273)	+0.012 (+0.62, 0.237)	+0.018 (+0.83, 0.139)
	GraphCross − Plain	+0.016 (+0.85, 0.131)	+0.009 (+0.46, 0.366)	+0.012 (+0.49, 0.333)	+0.010 (+0.80, 0.147)
	NodeCross − NodeRes	+0.006 (+0.26, 0.588)	+0.009 (+0.69, 0.195)	-0.001 (-0.13, 0.787)	+0.003 (+0.33, 0.503)
	GraphCross − GraphRes	-0.003 (-0.30, 0.545)	+0.002 (+0.08, 0.874)	+0.000 (+0.00, 0.999)	-0.008 (-0.32, 0.513)
	GraphRes − NodeRes	+0.015 (+0.87, 0.125)	-0.003 (-0.27, 0.583)	-0.003 (-0.18, 0.714)	+0.005 (+0.23, 0.637)
	NodeCross − GraphCross	-0.005 (-0.35, 0.476)	+0.010 (+0.50, 0.324)	+0.001 (+0.10, 0.832)	+0.007 (+0.49, 0.334)

Table 14. Residual-routing and parameter-sweep summary on four representative targets (transposed). Numbers are five-fold mean accuracy ± standard deviation. Bold: best per column.

Routing	AIDS+GraphRes+GIN	PROT+GraphRes+SAGE	AIDS+NodeCross+GAT	DD+NodeCross+GAT
Learnable	**0.9500 ± 0.0087**	0.7178 ± 0.0411	0.8970 ± 0.0491	0.7080 ± 0.0172
Top-k 0.25	0.9075 ± 0.0237	0.7241 ± 0.0407	0.9160 ± 0.0128	0.7062 ± 0.0249
Top-k 0.50	0.9330 ± 0.0251	0.7151 ± 0.0397	0.8960 ± 0.0483	0.7029 ± 0.0160
Top-k 0.75	0.9240 ± 0.0287	0.7214 ± 0.0318	0.8960 ± 0.0489	0.7164 ± 0.0248
Sparse 0.02	0.9470 ± 0.0181	0.7160 ± 0.0248	0.9060 ± 0.0268	0.7147 ± 0.0259
Sparse 0.05	0.9250 ± 0.0152	**0.7259 ± 0.0475**	**0.9180 ± 0.0162**	**0.7181 ± 0.0196**
Sparse 0.10	0.8315 ± 0.0549	0.7088 ± 0.0521	0.9025 ± 0.0139	0.7105 ± 0.0243
Best	Learnable	Sparse 0.05	Sparse 0.05	Sparse 0.05

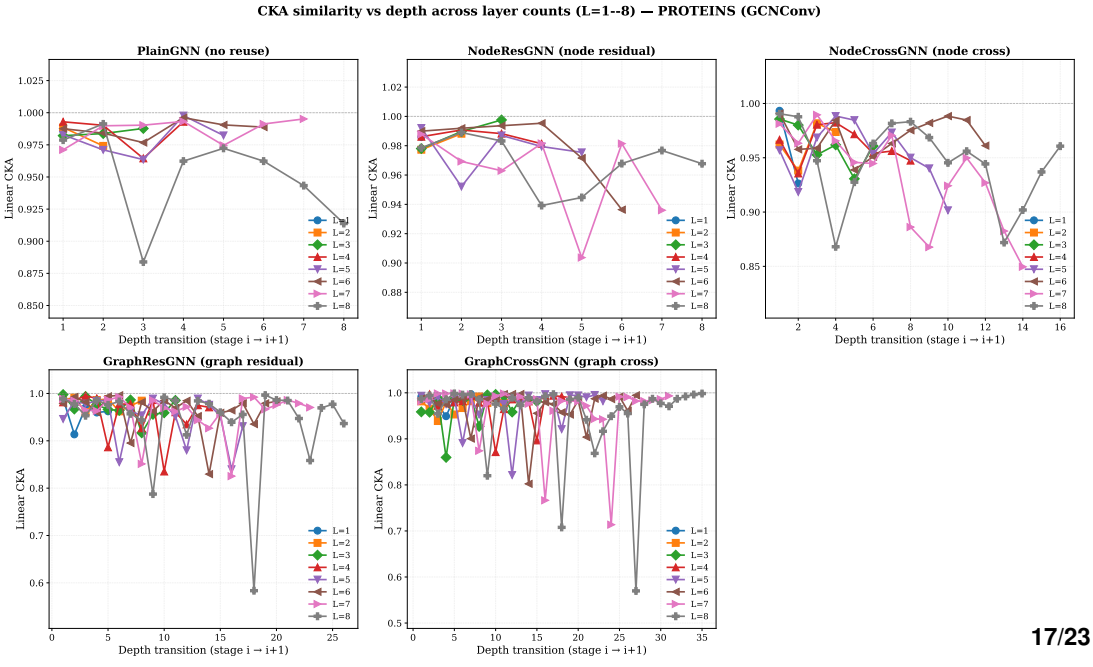

Figure 6. CKA similarity between adjacent depth stages across layer counts $L = 1$ through $L = 8$. Each subplot shows one model; curves correspond to different layer counts. Higher CKA indicates greater

Table 15. Family-wise routing winners.

Target	Best learnable	Best top-k	Best sparse	Overall winner
AIDS + GraphRes + GIN	0.9500	0.9330	0.9470	Learnable
PROTEINS + GraphRes + SAGE	0.7178	0.7241	0.7259	Sparse
AIDS + NodeCross + GAT	0.8970	0.9160	0.9180	Sparse
DD + NodeCross + GAT	0.7080	0.7164	0.7181	Sparse

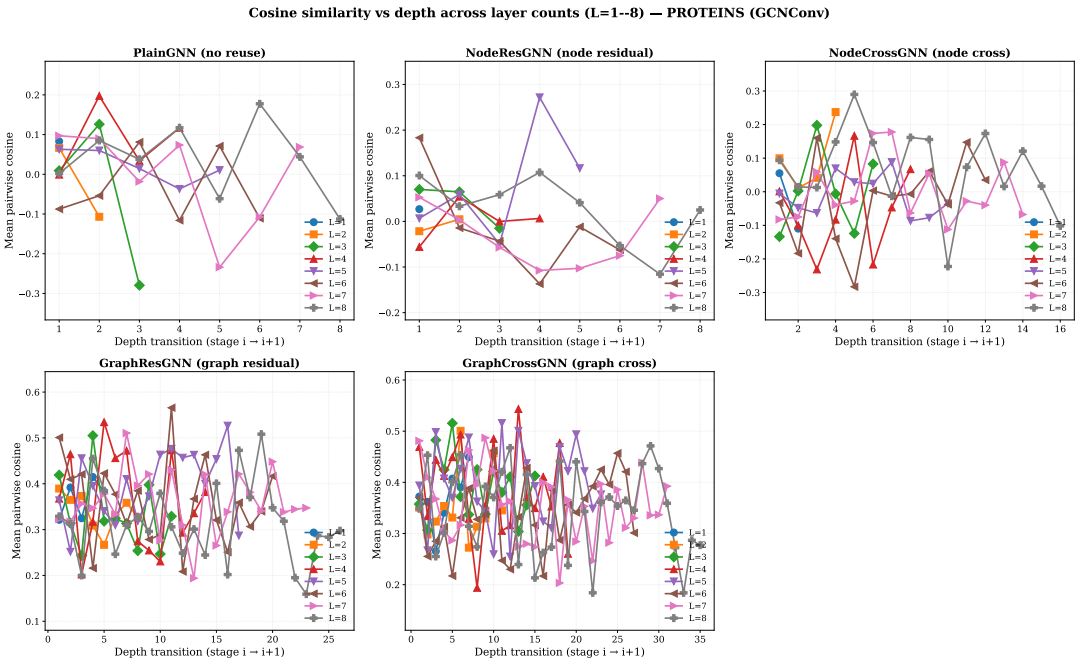

Figure 7. Cosine similarity between adjacent depth stages across layer counts $L = 1$ through $L = 8$. The pattern mirrors the CKA results: deeper stacking reduces adjacent-stage cosine similarity across all architectures, with PlainGNN showing the steepest decline at higher layer counts. Negative values indicate anti-correlated representations between adjacent stages.

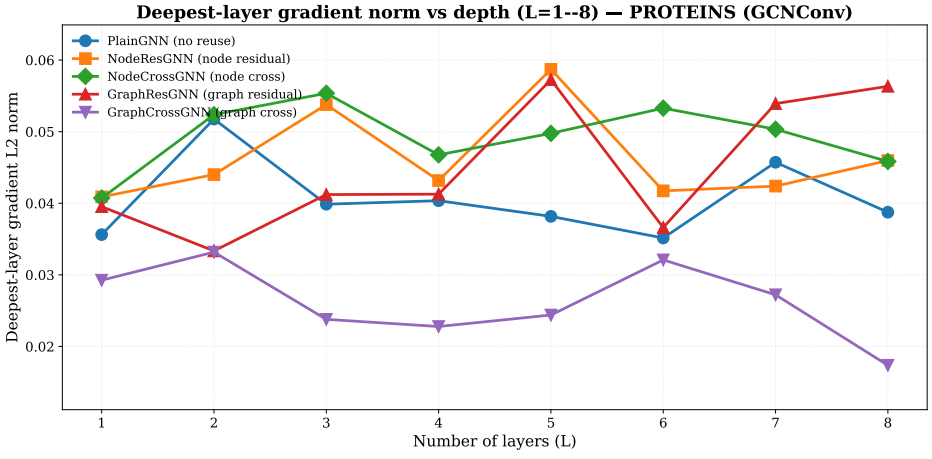

Figure 8. Deepest-layer gradient L2 norm as a function of layer count ($L = 1$ through $L = 8$) for all five architectures on PROTEINS (GCNConv). The residual family maintains more stable gradient signals at depth, while PlainGNN shows the steepest decline. Graph-level variants occupy an intermediate position.

Exploratory sensitivity analysis

Auxiliary sensitivity checks over depth, dropout, and learning rate on PROTEINS, DD, and ENZYMES do not materially change the main conclusion. On PROTEINS and DD, moderate tuning changes affect the gap between the two cross variants, but they do not overturn the broader conclusion that residual baselines remain the stronger default reference. On ENZYMES, the validation signal is visibly noisier, and promising local settings did not reproduce the original full-suite cross results under the final 5-fold average. These checks are reported for transparency, but they should not be read as evidence at the same level as the repeated-split benchmark and targeted ablations.

DISCUSSION

The main empirical message of this study is that the CR-GNN design space reveals systematic trade-offs among peak accuracy, cross-dataset consistency, and representational continuity—trade-offs that are not visible when architectures are compared only by a single winner-take-all metric. GraphResGNN wins most frequently (50% of combinations) but with the weakest representational continuity at depth and with outlier performance on specific datasets. NodeResGNN achieves the best representational continuity (CKA 0.968 at $L = 8$) but with the greatest performance volatility. Cross-residual architectures (NodeCrossGNN, GraphCrossGNN) occupy the middle ground in both dimensions, with GraphCrossGNN emerging as the most rank-consistent design. The contribution of the paper is therefore not a declaration of a single best architecture, but the controlled comparison framework that makes these trade-offs quantitatively visible.

Two paths to strong performance

The data support the existence of two distinct pathways to strong graph classification performance within the CR-GNN design space.

Path 1: Aggressive representational reshaping with graph-level context (GraphResGNN). GraphResGNN chains three graph-conditioned blocks, each of which receives the previous block's pooled graph embedding as a conditioning signal. This design produces the lowest layer-to-layer CKA values (0.942 at $L = 8$)—indicating substantial representational change between stages—yet achieves the highest raw accuracy on the majority of dataset–operator combinations. The mechanism appears to be that reinjecting graph-level context at each block creates a deliberate, structured reshaping of the representation: the model does not aim to preserve representations across depth, but rather to progressively refine them using accumulating global information. This strategy is most effective when global graph-level context is strongly discriminative (e.g., AIDS, ENZYMES, PROTEINS), but it can fail when local structural motifs are the dominant signal (e.g., MUTAG, where GraphResGNN ranks fifth).

Path 2: Balanced representational continuity with cross-branch exchange (GraphCrossGNN, NodeCrossGNN). Cross-residual architectures maintain intermediate CKA values while achieving the most consistent ranking across datasets. GraphCrossGNN, in particular, never ranks below fourth on any dataset (rank std 0.75), making it the safest architectural choice when dataset characteristics are uncertain or when stable performance across diverse tasks is valued over peak single-dataset accuracy. NodeCrossGNN's dominance on DD (3/4 operators) further demonstrates that dual-branch node-level exchange is specifically effective on large, sparse protein graphs where complementary parallel pathways can preserve discriminative structural signals that a single branch would suppress. The cross-residual pathway thus provides a form of "insurance" against the brittleness that can accompany aggressive representational reshaping.

The existence of these two paths carries a practical implication: the choice between GraphResGNN and a cross-residual variant should be guided by whether the practitioner prioritizes peak single-task accuracy (choose GraphResGNN) or consistent performance across diverse, possibly heterogeneous datasets (choose GraphCrossGNN), or specifically works with large sparse graphs and attention-based operators (choose NodeCrossGNN).

Node-level and graph-level reuse create different trade-off profiles

The distinction between node-level and graph-level reuse is practically important and interacts with the trade-off framework. GraphResGNN and GraphCrossGNN carry more parameters than their node-level counterparts (e.g., GraphResGNN ≈51k vs. NodeResGNN ≈23k on PROTEINS), yet this parameter advantage does not translate uniformly: GraphResGNN ranks first on PROTEINS and ENZYMES but fifth on MUTAG, where lighter node-level models perform better. This suggests that information-flow

topology, rather than parameter count, is the primary determinant of the performance–consistency trade-off. NodeCrossGNN, despite using fewer parameters than GraphResGNN, is the second-most winning architecture (6/24 combinations), and it achieves this through topology rather than capacity.

Why ENZYMES remains difficult

All models perform poorly on ENZYMES in absolute terms: the best model (GraphResGNN, 0.333) is only $\sim$16 percentage points above the 6-class random baseline (16.7%). Several factors contribute: only 600 graphs with 3-dimensional input features; a 6-class structure creating a harder decision boundary than the binary tasks dominating the rest of the benchmark; and the largest fold-level standard deviation ($\pm$0.100) in the biological core. The ENZYMES results are best interpreted as evidence that the architecture ranking remains consistent even under noisy, low-data regimes, rather than as evidence that any architecture solves the task well.

Residual routing and sparsity modulate the trade-off

The routing analysis reveals that routing strategy can shift the performance–consistency trade-off. Sparse routing with moderate threshold (0.05) is optimal on three of four representative targets, and two of these three targets involve cross-residual architectures. This points to a mechanism-level synergy: sparsity suppresses weak residual channels that would otherwise accumulate as noise across depth, while the cross-branch pathway ensures that genuinely complementary information is preserved through an alternative route. Learnable dense routing remains best on the strongest graph-level residual target (AIDS + GraphResGNN + GINConv), where rich historical context benefits from unfiltered reuse. The practical guidance is: combine sparse routing with cross-residual architectures for balanced, consistent performance; use learnable dense routing with graph-level residual architectures when maximizing peak accuracy on well-characterized datasets.

Depth-scaling evidence and the CKA–performance paradox

The depth-scaling analysis (Figures 6–8) documents a finding that challenges the intuitive assumption that stronger representational continuity should predict better performance. NodeResGNN achieves the highest CKA at $L = 8$ (0.968) but wins only 3/24 combinations with the greatest rank volatility. GraphResGNN achieves the lowest CKA (0.942) but wins 12/24 combinations. This "CKA–performance paradox" suggests that representational change between depth stages is not inherently detrimental—when structured through a mechanism such as graph-conditioned block reinjection, it can be productive. Conversely, preserving representations too faithfully (as NodeResGNN does) may limit the model's ability to adapt its representations to task-specific structure at each depth stage. The cross-residual variants occupy a productive middle ground: enough continuity to avoid the brittleness of PlainGNN, enough change to remain competitive with GraphResGNN.

Evidence hierarchy and study limitations

The strongest evidence comes from the five-fold benchmark and targeted five-fold ablations. The older single-fold sensitivity scans are useful only for optimization interpretation. A second limitation is statistical power: with five paired observations per combination, the tests have limited power to detect small-to-medium effects (Cohen's $d < 0.5$). This does not undermine the paper's conclusions, but it means that architecture comparisons should be understood as numerical orderings under controlled conditions rather than statements of statistical dominance. A third limitation is baseline scope: the benchmark includes representative baselines but is not an exhaustive leaderboard against every recent method. A fourth limitation is dataset scale: although biomolecular and meaningful, the datasets are medium-scale benchmarks. The study therefore supports conclusions about controlled graph-classification behavior and design trade-offs, not about large-scale scalability or direct biological discovery. A fifth limitation specific to the depth-scaling analysis is that it was conducted on a single dataset–operator combination (PROTEINS, GCNConv); extending it to other datasets and operators would test the generality of the CKA–performance paradox.

Future directions

The most important extension is to test whether the trade-off framework generalizes to larger graph benchmarks and richer biological inputs. The CKA–performance paradox—that the architecture with the weakest representational continuity achieves the highest peak accuracy—warrants further investigation

across diverse domains to determine whether it is a general property of deep GNN design spaces or specific to the biomolecular setting studied here. The finding that cross-residual plus sparse routing forms a particularly effective combination should also be tested on broader tasks where graph topology must be combined with stronger domain supervision.

CONCLUSIONS

This paper studies CR-GNN as a controlled biomolecular graph-classification framework and finds that the design space reveals systematic trade-offs among peak accuracy, cross-dataset consistency, and representational continuity. The main contribution is not a universally dominant architecture, but a reproducible comparison framework that makes these trade-offs quantitatively visible and provides practical guidance for architecture selection.

The key empirical findings are as follows. First, GraphResGNN achieves the highest peak accuracy most frequently (50% of 24 dataset–operator combinations) but with the weakest representational continuity at depth (CKA 0.942 at $L = 8$) and the greatest rank volatility (std 1.46). Second, NodeCrossGNN is the second-strongest architecture (25% of combinations), dominating DD across three of four operators, and is particularly effective on large sparse graphs with attention-based operators. Third, GraphCrossGNN is the most rank-consistent design (std 0.75), never falling below fourth place, making it the safest architectural choice when dataset characteristics are uncertain. Fourth, the depth-scaling analysis documents a "CKA–performance paradox": the architecture with the highest representational continuity (NodeResGNN, CKA 0.968) achieves the fewest wins among reuse variants, while the architecture with the lowest continuity (GraphResGNN) achieves the most. This challenges the assumption that stronger representational stability necessarily produces better performance.

The routing analysis provides mechanism-level evidence that sparse routing combined with cross-residual architectures forms a particularly effective configuration, with sparse mode (threshold 0.05) optimal on three of four representative targets. This synergy—sparsity suppressing weak channels while cross-branch exchange preserves complementary information—represents a practical design principle for future GNN architecture development.

The scope of these conclusions should remain disciplined. The strongest claims are supported by the stratified five-fold benchmark, the targeted five-fold ablations, and the depth-scaling analysis ($L = 1$–8) on PROTEINS under GCNConv. The statistical evidence (Tables 12 and 13) confirms that architecture-level effects are modest relative to fold-level variation, consistent with the paper's mechanism-oriented rather than leaderboard-oriented framing. The study addresses controlled graph-classification behavior on medium-scale biomolecular benchmarks and does not by itself establish large-scale scalability or direct biological interpretability.

In practical terms, this work provides the following guidance: choose GraphResGNN with learnable dense routing when peak accuracy on well-characterized datasets is the priority; choose GraphCrossGNN with sparse routing when consistent, predictable performance across diverse datasets is required; choose NodeCrossGNN with GATConv when working with large, sparse graphs where dual-branch complementary pathways are valuable. These results position CR-GNN as a practical design space and provide a methodological basis for future work on richer biological inputs, larger benchmarks, and graph-learning settings where the trade-offs documented here can be further tested and refined.

ACKNOWLEDGMENTS

The authors thank the open-source graph learning community and the maintainers of the benchmark resources used in this study. They also thank the reviewers and editors for their time and feedback.

REFERENCES

Uri Alon and Eran Yahav. On the bottleneck of graph neural networks. In *International Conference on Learning Representations (ICLR)*, 2021.

Xavier Bresson and Thomas Laurent. Residual gated graph convnets. *arXiv preprint arXiv:1711.07553*, 2017.

Tianle Cai, Jiacheng Luo, Sheng Wang, Kai Zhang, Yu Tian, Liwei Yang, Zekun Li, and Kilian Weinberger. Graphnorm: A principled approach to accelerating graph neural network training. In *International Conference on Machine Learning*, pages 1277–1287, 2021.

Ming Chen, Zhewei Wei, Zengfeng Huang, Bolin Ding, and Yaliang Li. Simple and deep graph convolutional networks. In *International Conference on Machine Learning (ICML)*, pages 1725–1735, 2020a.

Yuning Chen, Xihao Wei, Pan Xu, Xinwen Wang, Ping Luo, Zhiyuan Li, and Jian Tang. Revisiting over-smoothing in deep gcns. *arXiv preprint arXiv:2003.13663*, 2020b.

Matthias Fey and Jan Eric Lenssen. Fast graph representation learning with pytorch geometric. *arXiv preprint arXiv:1903.02428*, 2019.

Hongyang Gao and Shuiwang Ji. Graph u-net. *arXiv preprint arXiv:1905.05178*, 2019.

Justin Gilmer, Samuel S Schoenholz, Patrick F Riley, Oriol Vinyals, and George E Dahl. Neural message passing for quantum chemistry. In *International Conference on Machine Learning*, pages 1263–1272, 2017.

William L Hamilton, Rex Ying, and Jure Leskovec. Inductive representation learning on large graphs. In *Advances in neural information processing systems*, pages 1024–1034, 2017.

Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016.

Thomas N Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. *arXiv preprint arXiv:1609.02907*, 2016.

Johannes Klicpera, Alan Bojchevski, and Stephan Günnemann. Combining label propagation and simple models out-performs graph neural networks. *arXiv preprint arXiv:1810.05997*, 2018.

Guohao Li, Elias Muller, Abbas Thabet, and Bernard Ghanem. Deeper insights into graph convolutional networks for semi-supervised learning. *arXiv preprint arXiv:1809.02584*, 2018.

Christopher Morris, Roger Wattenhofer, and Kristian Kersting. Tudataset: A collection of benchmark datasets for learning with graphs. *arXiv preprint arXiv:2007.08663*, 2020.

Kenta Oono and Taiji Suzuki. Simple and deep graph convolutional networks. *arXiv preprint arXiv:2006.13604*, 2020.

Yu Rong, Wenbing Huang, Tingyang Xu, and Junzhou Huang. Dropedge: Towards deep graph convolutional networks on node classification. *arXiv preprint arXiv:1907.10903*, 2019.

Jake Topping, Francesco Di Giovanni, Benjamin Chamberlain, Michael Bronstein, Douwe Vendrig, and Pietro Lio. Understanding over-squashing and bottlenecks on graphs via curvature. *arXiv preprint arXiv:2111.14522*, 2021.

Zonghan Wu, Shirui Pan, Fengwen Chen, Guoding Long, Cheng Zhang, and Sheng Yu Zhang. A comprehensive survey on graph neural networks. *IEEE Transactions on Neural Networks and Learning Systems*, 32(1):4–24, 2021.

Keyulu Xu, Chengtao Li, Yonglong Tian, Xiao Du, Jure Leskovec, and Stefanie Jegelka. Representation learning on graphs with jumping knowledge networks. In *International Conference on Machine Learning*, pages 5449–5458, 2018.

Rex Ying, Jiaxuan You, Christopher Morris, Xiang Ren, William L Hamilton, and Jure Leskovec. Hierarchical graph representation learning with differentiable pooling. In *Advances in neural information processing systems*, pages 4800–4810, 2018.

Lingxiao Zhao and Leman Akoglu. Pairnorm: Tackling over-smoothing in gns. *arXiv preprint arXiv:2009.10698*, 2020.

Jie Zhou, Ganqu Cui, Zhengyan Zhang, Cheng Yang, Zhaocheng Liu, Zhongyuan Wang, Peng Li, Ming Sun, Yu Shi, et al. Graph neural networks: A review of methods and applications. *AI Open*, 1:57–81, 2020.